
UniChat

Documentazione funzionale e tecnica

Relazione di progetto per web app universitaria realtime

UniChat è una piattaforma web per la comunicazione universitaria. Il progetto integra autenticazione, profili utente con immagine, server tematici, link di invito, canali, messaggistica persistente, aggiornamenti in tempo reale con Socket.IO, pannello amministrativo e un gioco privato del Tris tra utenti.

Progetto UniChat

1 settembre 2026

Indice

1	Introduzione	1
2	Utilizzo della web app lato utente	1
2.1	Accesso iniziale	1
2.2	Registrazione di un nuovo utente	1
2.3	Immagine profilo	2
2.4	Navigazione tra server e canali	2
2.5	Invito tramite link	2
2.6	Creazione di un server	2
2.7	Invio e lettura dei messaggi	2
2.8	Stato della connessione	3
2.9	Gioco privato del Tris	3
2.10	Pannello amministrativo	3
3	Caratteristiche funzionali	3
3.1	Autenticazione utenti	3
3.2	Università	3
3.3	Server	4
3.4	Canali	4
3.5	Messaggi	4
4	Architettura del progetto	4
5	Front-end	5
5.1	Struttura HTML	5
5.2	Stile grafico	5
5.3	Logica JavaScript	5
6	Back-end	5
6.1	Node.js ed Express	6
6.2	Configurazione	6
6.3	Controller, route e middleware	6
7	Database	7
7.1	Tecnologie	7
7.2	Modello dati	7

8	Sicurezza	7
8.1	Password	7
8.2	Token di sessione	7
8.3	Dati applicativi	8
8.4	Admin	8
9	Realtime con Socket.IO	8
10	Docker e ambiente locale	8
11	Scelte progettuali	9
11.1	Separazione front-end/back-end	9
11.2	REST per operazioni persistenti	9
11.3	Socket.IO per realtime	9
11.4	PostgreSQL e Prisma	9
11.5	Tris in memoria	10
11.6	Admin con codice di setup	10
12	Limiti attuali e possibili estensioni	10
13	Conclusione	10

1 Introduzione

UniChat è una web app di chat universitaria ispirata alle funzioni principali di Discord, ma adattata a un contesto didattico e accademico. L'obiettivo è fornire uno spazio organizzato in cui studenti e docenti possano creare server collegati a una università, discutere in canali tematici, scambiarsi messaggi in tempo reale e usare una piccola area privata per giocare a Tris.

Il progetto è stato mantenuto volutamente essenziale: non include notifiche complesse, chiamate vocali reali, file sharing, sistemi di moderazione avanzati o giochi multipli. Questa scelta rende il codice più leggibile e adatto a mostrare in modo chiaro l'integrazione tra front-end, back-end, REST API, database relazionale, autenticazione e WebSocket.

Obiettivo della relazione

Questa relazione descrive sia l'utilizzo pratico della web app dal punto di vista dell'utente, sia l'architettura tecnica che permette al sistema di funzionare. La prima parte è quindi funzionale; la seconda entra nelle scelte implementative.

2 Utilizzo della web app lato utente

2.1 Accesso iniziale

All'apertura dell'app, l'utente visualizza la schermata di autenticazione. Da qui può scegliere se accedere con un account esistente oppure creare un nuovo profilo. L'interfaccia distingue chiaramente le due operazioni tramite due schede: **Login** e **Registrati**.

Per accedere sono richiesti email e password. Dopo il login, la web app carica automaticamente i server dell'utente, i server disponibili nella sua università e lo stato della connessione realtime.

2.2 Registrazione di un nuovo utente

La registrazione richiede:

- nome completo;
- indirizzo email;
- password;
- università di appartenenza, obbligatoria solo per chi non usa un invito;
- immagine profilo opzionale tramite URL;
- eventuale codice admin, se l'account deve essere creato con privilegi amministrativi.

Se l'università non è ancora presente nella lista, l'utente può aggiungerla direttamente dal form indicando nome e dominio. Se invece l'utente arriva tramite link di invito, può creare l'account senza specificare l'università: l'invito è sufficiente per collegarlo al server.

2.3 Immagine profilo

Durante la registrazione l'utente può indicare un URL per la propria immagine profilo. L'immagine viene mostrata nei messaggi e nella lista membri del server. Se l'immagine non è presente o non è raggiungibile, la web app usa automaticamente le iniziali del nome come fallback.

2.4 Navigazione tra server e canali

Dopo l'accesso, l'utente entra nell'area principale dell'app. A sinistra è presente la barra dei server, mentre nella colonna laterale vengono mostrati i server disponibili, i canali del server selezionato e gli utenti con cui è possibile avviare una partita privata.

Un server rappresenta uno spazio di discussione, ad esempio un corso, un gruppo di studio o un dipartimento. Se l'utente non appartiene ancora a un server della propria università, può entrare dalla sezione **Server disponibili**. Una volta entrato, il server compare nella barra laterale e diventa selezionabile.

2.5 Invito tramite link

Ogni server possiede un codice invito univoco. Un membro può copiare il link del server attivo e condividerlo con un'altra persona. Chi riceve il link può entrare nel server anche se non appartiene alla stessa università del proprietario del server.

Se il destinatario non ha ancora effettuato l'accesso, l'app conserva l'invito e lo applica dopo il login. Se deve registrarsi, può farlo senza selezionare una università perché l'invito identifica già il server di destinazione.

2.6 Creazione di un server

L'utente autenticato può creare un nuovo server tramite il pulsante dedicato. Deve indicare un nome e, opzionalmente, una descrizione. Il creatore diventa automaticamente proprietario e membro del server.

Alla creazione vengono aggiunti tre canali iniziali:

- **generale**, per la conversazione principale;
- **materiali**, per argomenti legati allo studio;
- **studio-vocale**, canale vocale rappresentato nell'interfaccia.

2.7 Invio e lettura dei messaggi

Quando l'utente seleziona un canale testuale, l'app mostra lo storico dei messaggi già presenti nel database. In fondo alla pagina è disponibile il campo di composizione. Dopo l'invio, il messaggio viene salvato sul server e propagato in tempo reale agli altri utenti collegati allo stesso canale.

Il limite massimo di un messaggio è di 1000 caratteri. Questo limite evita input eccessivamente lunghi e mantiene la conversazione leggibile.

2.8 Stato della connessione

Nell'header della chat è presente un indicatore **online/offline**. Questo elemento segnala lo stato della connessione Socket.IO:

- **online**: il browser è collegato al server realtime;
- **offline**: la connessione realtime è assente o interrotta.

2.9 Gioco privato del Tris

La sezione **Privati** permette di selezionare un altro membro del server e proporgli una partita a Tris. L'altro utente può accettare o rifiutare l'invito. Una volta iniziata la partita, le mosse vengono sincronizzate tramite Socket.IO.

Il Tris è una funzione temporanea e non persistente: se il server viene riavviato, le partite in corso vengono perse. Questa scelta è coerente con la natura accessoria del gioco rispetto alla chat.

2.10 Pannello amministrativo

Gli utenti con ruolo **ADMIN** visualizzano un pulsante **Admin**. Il pannello amministrativo consente di consultare utenti, server e ultimi messaggi, oltre a eliminare utenti, server o messaggi quando necessario.

La protezione non è affidata al solo front-end: anche se un utente modificasse manualmente l'interfaccia del browser, il backend controllerebbe comunque il ruolo prima di consentire operazioni amministrative.

3 Caratteristiche funzionali

3.1 Autenticazione utenti

L'app permette la registrazione e il login degli utenti. Ogni utente fornisce nome completo, email e password. L'università è richiesta per la registrazione ordinaria, mentre può essere omessa quando l'utente possiede un invito valido. Dopo il login, il server crea un token di sessione che viene salvato nel browser tramite `localStorage` e inviato nelle richieste HTTP con header `Authorization: Bearer`.

Le password non vengono salvate in chiaro. Il backend salva solo un hash bcrypt nel campo `User.passwordHash`. Questo significa che la password originale non è recuperabile dal database. L'immagine profilo è invece salvata come URL opzionale nel campo `User.avatarUrl`.

3.2 Università

Gli utenti sono collegati a una università. La web app mostra l'elenco delle università disponibili durante la registrazione e permette anche di crearne una nuova indicando nome e dominio.

La scelta di legare utenti e server a una università permette di separare logicamente gli ambienti: un utente vede e può entrare nei server disponibili della propria università, non in quelli di altri contesti.

3.3 Server

Un server rappresenta uno spazio di discussione, ad esempio un corso o un gruppo universitario. Un utente autenticato può creare un server indicando nome e descrizione. Il creatore diventa automaticamente membro e proprietario del server.

Quando viene creato un server, il backend genera anche un codice invito univoco e crea tre canali iniziali: **generale**, **materiali** e **studio-vocale**.

3.4 Canali

Ogni server contiene canali. I canali possono essere testuali, per messaggi persistenti, oppure vocali, rappresentati come spazio dedicato nell'interfaccia ma senza implementare audio reale.

Gli utenti membri del server possono creare nuovi canali indicando nome e tipo. I nomi dei canali vengono normalizzati lato backend in minuscolo e con trattini al posto degli spazi.

3.5 Messaggi

Nei canali testuali gli utenti possono inviare messaggi. Il messaggio viene validato, troncato a 1000 caratteri, salvato nel database e poi inviato in tempo reale agli utenti collegati allo stesso canale.

Lo storico dei messaggi viene recuperato via REST quando l'utente apre un canale. La sincronizzazione immediata dei nuovi messaggi avviene invece con Socket.IO.

4 Architettura del progetto

Il progetto è diviso in tre aree principali:

- **frontend/**, che contiene HTML, CSS e JavaScript del browser;
- **backend/**, che contiene server Node.js, Express, Prisma e Socket.IO;
- **docs/**, che contiene documentazione tecnica e relazione PDF.

```
UniChat/  
  frontend/  
    index.html  
    assets/  
      css/styles.css  
      js/app.js  
  backend/  
    prisma/  
      schema.prisma  
      migrations/  
  src/  
    config/  
    controllers/  
    lib/  
    middleware/  
    routes/  
    sockets/
```

```
    utils/  
    app.js  
    main.js  
docs/  
    architettura-unichat.md  
    UniChat_documentazione.tex  
docker-compose.yml  
README.md  
Leggimi.txt
```

5 Front-end

5.1 Struttura HTML

Il file `frontend/index.html` definisce la struttura della pagina. Contiene form di login e registrazione, selezione università, area server, lista canali, lista membri, area messaggi, pannello Tris, pannello admin e dialog per la creazione dei server.

5.2 Stile grafico

Il file `frontend/assets/css/styles.css` gestisce layout e aspetto grafico. Le scelte principali sono:

- CSS Grid per la struttura generale a colonne;
- Flexbox per header, liste e azioni;
- variabili CSS in `:root` per colori, bordi, ombre e testi;
- media query per adattare la UI sotto gli 850px;
- classi di stato come `hidden`, `active`, `online`;
- messaggi e indicatore online/offline allineati a destra.

La UI è composta da una barra server a sinistra, una sidebar di lavoro e una grande area centrale per chat, Tris o pannello admin.

5.3 Logica JavaScript

Il file `frontend/assets/js/app.js` contiene la logica client. Mantiene uno stato globale con token utente, utente autenticato, server dell'utente, server disponibili, server e canale attivi, utente privato selezionato, stato della partita Tris e socket Socket.IO.

Il client usa `fetch` per le API REST e Socket.IO per eventi realtime. Le funzioni principali renderizzano server, canali, messaggi, pannello admin e partita Tris senza framework esterni.

6 Back-end

6.1 Node.js ed Express

Il backend usa Node.js con Express. La scelta di Express è adatta a un progetto didattico perché rende espliciti route, middleware e controller senza introdurre troppa astrazione.

Il file `backend/src/app.js` crea l'app Express, abilita CORS, JSON parsing, file statici e route API. Il file `backend/src/main.js` crea il server HTTP, collega Socket.IO e avvia l'ascolto sulla porta configurata.

6.2 Configurazione

La configurazione è centralizzata in `backend/src/config/env.js`. Il file legge `backend/.env` e fornisce:

- `PORT`;
- `CORS_ORIGIN`;
- `SESSION_DURATION_DAYS`;
- `SALT_ROUNDS`;
- `SESSION_TOKEN_PEPPER`;
- `ADMIN_SETUP_CODE`.

La scelta di caricare esplicitamente `backend/.env` rende l'avvio meno fragile: il server trova le variabili anche se Node viene lanciato da una cartella diversa.

6.3 Controller, route e middleware

Area	Responsabilità
<code>authControllers.js</code>	Registrazione, login, logout e recupero dell'utente autenticato.
<code>universityControllers.js</code>	Lista, creazione ed eliminazione delle università.
<code>serverControllers.js</code>	Gestione server, canali e ingresso nei server esistenti.
<code>messageControllers.js</code>	Lettura e creazione dei messaggi.
<code>adminControllers.js</code>	Panoramica admin e cancellazioni protette.
<code>authMiddleware.js</code>	Validazione del bearer token e valorizzazione di <code>req.user</code> .
<code>adminMiddleware.js</code>	Controllo del ruolo ADMIN.

Le route sono separate dai controller e sono raccolte sotto percorsi API distinti:

- `/api/auth`;
- `/api/universities`;
- `/api/servers`;
- `/api/channels`;
- `/api/admin`.

Questa organizzazione rende più leggibile il progetto e semplifica l'estensione futura.

7 Database

7.1 Tecnologie

Il progetto usa PostgreSQL come database relazionale e Prisma come ORM. PostgreSQL è adatto per dati strutturati con relazioni chiare. Prisma semplifica query e migrazioni mantenendo uno schema leggibile.

7.2 Modello dati

Modello	Responsabilità
University	Rappresenta una università con nome e dominio unici.
User	Rappresenta un account con email, hash password, nome, immagine profilo opzionale, ruolo e università opzionale.
Session	Rappresenta una sessione autenticata con token hash/HMAC e scadenza.
Server	Rappresenta uno spazio di discussione legato a una università, a un proprietario e a un codice invito.
ServerMember	Tabella ponte tra utenti e server. Impedisce doppie iscrizioni con vincolo unico.
Channel	Rappresenta un canale testuale o vocale dentro un server.
Message	Rappresenta un messaggio persistente legato a utente e canale.

Nel database vengono salvati utenti, URL delle immagini profilo, università, sessioni, server, codici invito, appartenenze ai server, canali e messaggi. Non vengono salvate le partite di Tris: le partite vivono in una Map in memoria dentro il processo Socket.IO.

8 Sicurezza

8.1 Password

Le password sono protette con bcrypt. Durante la registrazione il backend riceve la password, genera un hash con un numero di round configurabile e salva solo l'hash. Durante il login confronta la password inserita con l'hash salvato.

Questa è la scelta corretta per le password: non devono essere cifrate in modo reversibile, ma hashate con un algoritmo pensato per resistere ad attacchi brute force.

8.2 Token di sessione

Il token reale viene generato con `crypto.randomBytes(32)` e inviato al client al login. Nel database non viene salvato il token in chiaro: viene salvato un HMAC SHA-256 calcolato con `SESSION_TOKEN_PEPPER`.

Questa scelta aggiunge un segreto lato server. Se qualcuno ottiene solo un dump del database, non può verificare facilmente token candidati senza conoscere anche il pepper.

8.3 Dati applicativi

I dati applicativi, come email, nomi, URL avatar, università, server, canali e contenuto dei messaggi, restano leggibili nel database. Questa è una scelta consapevole per mantenere il progetto semplice e interrogabile.

Per cifrare anche questi campi servirebbe una cifratura applicativa campo per campo, ad esempio AES-GCM, con chiavi conservate fuori dal database. Questo aumenterebbe la complessità per ricerca, ordinamento, debug e gestione delle chiavi.

8.4 Admin

Il ruolo **ADMIN** è controllato lato backend. Le API admin richiedono token valido, sessione non scaduta e utente con ruolo **ADMIN**. Il pannello admin nel front-end è solo una comodità visuale; la protezione reale è nel backend.

9 Realtime con Socket.IO

Socket.IO viene usato per due aree: chat in tempo reale e Tris privato.

Quando il browser si collega, il client invia l'evento **authenticate** con il token. Il backend associa il socket all'utente e lo inserisce in una stanza privata **user:<id>**. I canali usano invece stanze identificate dal **channelId**.

Quando un messaggio viene inviato:

1. il client emette **send_message**;
2. il backend valida token, utente, canale e permessi;
3. il backend salva il messaggio nel database;
4. il backend emette **message_created** nella stanza del canale;
5. i client connessi aggiornano la UI.

Gli eventi Socket.IO principali del Tris sono **private_game_request**, **private_game_accept**, **private_game_move** e **private_game_restart**.

10 Docker e ambiente locale

Docker è usato per avviare PostgreSQL senza installarlo manualmente nel sistema. La configurazione si trova nel file **docker-compose.yml**. Il servizio principale è **postgres** e contiene:

- immagine **postgres:15-alpine**;
- utente **unichat_user**;
- password **unichat_password**;
- database **unichat_db**;
- porta locale 5433;
- volume persistente **postgres_data**.

Il backend si collega al database tramite:

```
postgresql://unichat_user:unichat_password@localhost:5433/unichat_db?schema=public
```

Il flusso locale previsto è:

1. avviare Docker Desktop se si lavora da Windows/WSL;
2. avviare PostgreSQL con `docker compose up -d`;
3. entrare in `backend/`;
4. installare dipendenze con `npm install`;
5. generare Prisma con `npm run prisma:generate`;
6. applicare migrazioni con `npm run prisma:migrate`;
7. avviare il server con `npm run dev`.

L'app viene servita su:

`http://localhost:5000`

Se Docker non è installato o non è visibile in WSL, Prisma restituisce errore P1001 perché il database non è raggiungibile.

11 Scelte progettuali

11.1 Separazione front-end/back-end

Il front-end è separato dal back-end per rendere chiara la responsabilità dei file. Il browser gestisce DOM e interazione; il server gestisce dati, permessi, sessioni e realtime.

11.2 REST per operazioni persistenti

Le REST API sono usate per operazioni classiche: login, registrazione, server, canali e storico messaggi. Sono semplici da testare, leggibili e adatte alle operazioni CRUD.

11.3 Socket.IO per realtime

Socket.IO è usato quando serve aggiornamento immediato: nuovi messaggi e Tris. Usare solo REST avrebbe richiesto polling continuo; Socket.IO evita richieste ripetute inutili.

11.4 PostgreSQL e Prisma

PostgreSQL fornisce relazioni, vincoli e persistenza stabile. Prisma rende esplicito lo schema e semplifica migrazioni e query.

11.5 Tris in memoria

Il Tris non viene salvato nel database perché è una funzione temporanea. Salvare ogni partita avrebbe richiesto nuove tabelle, stato storico, pulizia e regole di recupero. Per un progetto didattico è più coerente tenerlo in memoria.

11.6 Admin con codice di setup

Il codice admin permette di creare un super user dalla web app senza scrivere manualmente SQL. Allo stesso tempo, il codice resta nel file `.env`, quindi non è esposto nel repository.

12 Limiti attuali e possibili estensioni

Il progetto funziona come base completa, ma ha limiti consapevoli:

- i canali vocali sono solo rappresentati, non implementano audio reale;
- il Tris non è persistente e si perde al riavvio del server;
- le immagini profilo sono gestite tramite URL esterno, non tramite upload locale;
- i messaggi non sono cifrati campo per campo;
- non esiste ancora gestione avanzata di permessi per proprietari e moderatori;
- il deploy su Netlify da solo non è ideale per Socket.IO;
- per più istanze backend servirebbe un adapter Socket.IO condiviso, ad esempio Redis.

Possibili evoluzioni:

- aggiungere ruoli per server;
- aggiungere cancellazione o modifica messaggi per autore;
- aggiungere upload immagini con storage dedicato;
- cifrare messaggi sensibili a livello applicativo;
- usare Redis per scalare Socket.IO;
- creare un deploy su Render o Railway con PostgreSQL gestito;
- aggiungere test automatici di API e socket.

13 Conclusione

UniChat è una web app full-stack organizzata per mostrare in modo chiaro come collaborano front-end, back-end, database e realtime. Le scelte tecniche privilegiano leggibilità, separazione delle responsabilità e funzionalità concrete: autenticazione, profili utente, inviti ai server, canali, messaggi persistenti, realtime, Tris privato e pannello admin.

Dal punto di vista dell'utente, l'app offre un flusso semplice: registrazione, ingresso nella propria università, creazione o selezione dei server, scelta dei canali e conversazione immediata. Dal punto

di vista tecnico, il progetto dimostra l'uso coordinato di Express, Prisma, PostgreSQL e Socket.IO in una struttura chiara e mantenibile.

Il progetto è adatto per essere eseguito localmente con PostgreSQL via Docker e può essere pubblicato su piattaforme che supportano un server Node persistente, come Render o Railway.